27. Microservices vs Monolith

Key Terms:

Term	Full Name	What it does
Monolith	Single deployment unit	All code in one deployable application
Microservice	Independent service	Small, focused service with own database
Bounded Context	DDD term	Logical boundary around a domain area
Service Mesh	Infrastructure layer	Handles service-to-service communication
Saga Pattern	Distributed transactions	Sequence of local transactions with compensations
Strangler Fig	Migration pattern	Gradually replace monolith with services
Conway's Law	Organization principle	System design mirrors team structure

Problem Statement

Decide when to use microservices architecture versus a monolith, and how to evolve between them.

Clarifying Questions to Ask

1. **Team size**: How many developers? How organized?
2. **Domain complexity**: How many bounded contexts?
3. **Scale requirements**: Different components scale differently?
4. **Deployment frequency**: How often do you deploy?
5. **Technology diversity**: Need different languages/frameworks?
6. **Operational maturity**: DevOps capabilities?
7. **Current state**: Greenfield or existing system?

API Design

API Gateway Endpoints (Microservices)

Method	Endpoint	Description
`ANY`	`/api/v1/{service}/**`	Route to appropriate microservice
`GET`	`/api/v1/registry/services`	List registered services
`GET`	`/api/v1/registry/services/{name}/health`	Service health check

Inter-Service Communication

Pattern	Protocol	Use Case
REST	HTTP/JSON	Synchronous request-response
gRPC	HTTP/2 + Protobuf	High-performance internal calls
Events	Kafka/Event Hubs	Async event-driven communication

Service Mesh APIs

Method	Endpoint	Description
`GET`	`/api/v1/mesh/topology`	Get service dependency graph
`GET`	`/api/v1/mesh/traffic`	Get inter-service traffic metrics
`PUT`	`/api/v1/mesh/routing/{service}`	Configure traffic splitting (canary)

API Versioning Strategy

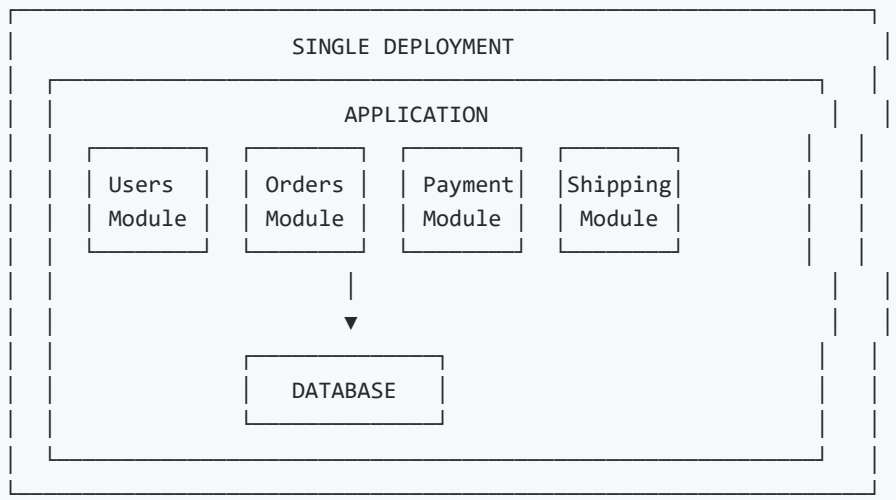
```
# URL versioning
/api/v1/users
/api/v2/users

# Header versioning
Accept: application/vnd.myapp.v2+json
```

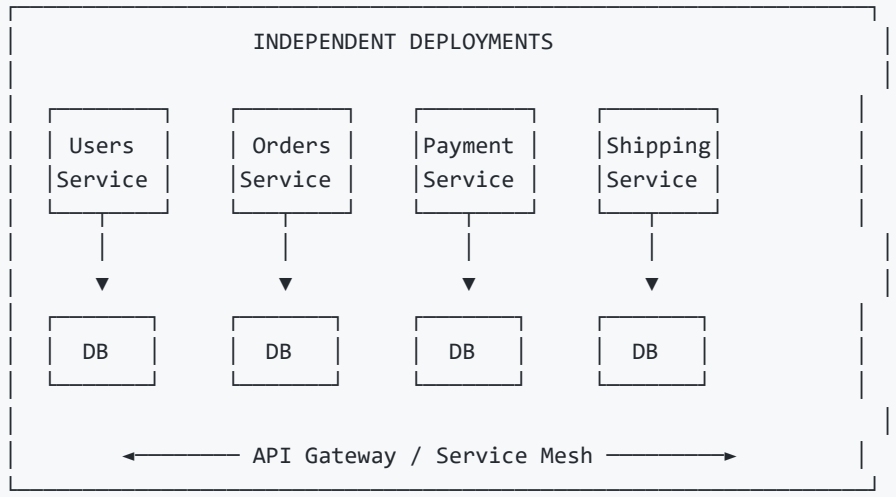
Comparison

```
┌─────────────────────────────────────────────────────────────┐
│              MONOLITH vs MICROSERVICES                      │
└─────────────────────────────────────────────────────────────┘
```

MONOLITH:



MICROSERVICES:



When to Choose What

CHOOSE MONOLITH WHEN:

```
✓ Small team (< 10 developers)
✓ New product with unclear requirements
✓ Simple domain
✓ Limited DevOps experience
✓ Need to move fast and iterate
✓ Tight budget (no infrastructure overhead)

"Start with a monolith, extract services when needed"
                        - Martin Fowler
```

CHOOSE MICROSERVICES WHEN:

```
✓ Large team (multiple teams, 50+ developers)
✓ Clear domain boundaries (bounded contexts)
```

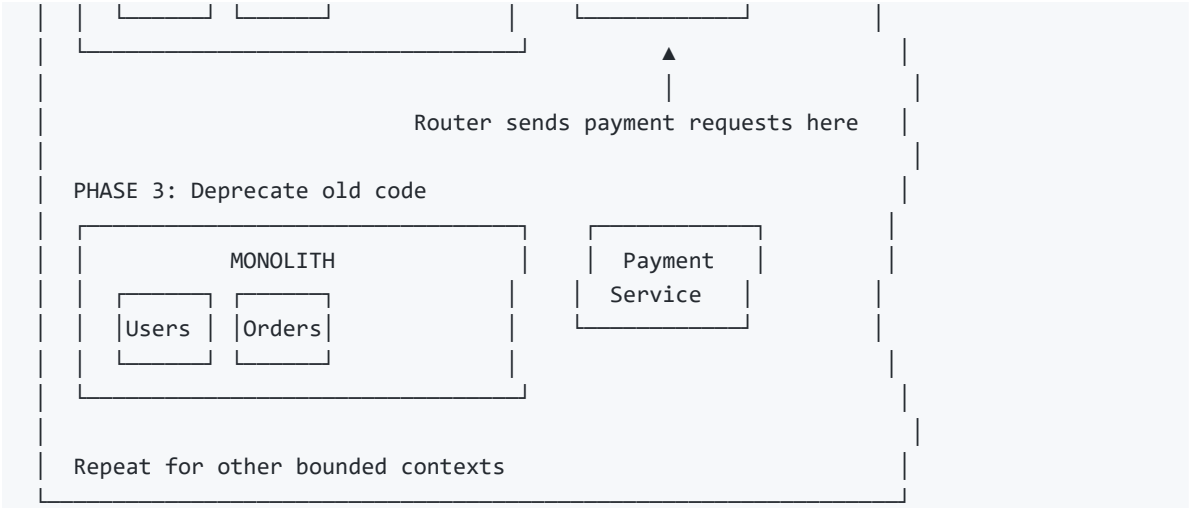
```
│  ✓ Different scaling requirements per component                │
│  ✓ Need independent deployments                                │
│  ✓ Different technology requirements                           │
│  ✓ Strong DevOps/platform team                                  │
│                                                                 │
│  "You must be this tall to ride" - organizational maturity     │
└─────────────────────────────────────────────────────────────────┘
```

Comparison Table

Aspect	Monolith	Microservices
Deployment	All-or-nothing	Independent
Scaling	Vertical or full horizontal	Per-service
Technology	Single stack	Polyglot
Data	Shared database	Database per service
Debugging	Simple (single process)	Complex (distributed tracing)
Latency	In-process calls	Network calls
Consistency	ACID transactions	Eventual consistency
Team autonomy	Low	High
Operational complexity	Low	High
Initial velocity	Fast	Slow (infrastructure setup)

Migration Strategy: Strangler Fig Pattern

```
┌─────────────────────────────────────────────────────────────────┐
│  STRANGLER FIG PATTERN: Gradually extract services              │
│                                                                 │
│  PHASE 1: Identify boundary                                     │
│  ┌──────────────────────────────────┐                           │
│  │           MONOLITH               │                           │
│  │  ┌───────┐ ┌───────┐ ┌───────┐   │                           │
│  │  │Users  │ │Orders │ │ Pay   │   │  ← Payment has clear      │
│  │  └───────┘ └───────┘ └───────┘   │    boundary, extract first │
│  └──────────────────────────────────┘                           │
│                                                                 │
│  PHASE 2: Create new service, route traffic                     │
│  ┌──────────────────────────────────┐    ┌──────────────┐       │
│  │           MONOLITH               │    │  Payment     │       │
│  │  ┌───────┐ ┌───────┐             │◄───┤  Service     │       │
│  │  │Users  │ │Orders │             │    │  (new)       │       │
```



Trade-offs

Decision	Monolith	Microservices	Recommendation
Startup	✓		**Monolith** - speed
Scale-up		✓	**Micro** - team autonomy
Complexity	Low	High	Consider team maturity
Data consistency	✓		**Monolith** if ACID critical

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Container orchestration for microservices	Tech Guide
Kafka	Event-driven communication between services	Tech Guide
Circuit Breaker	Fault tolerance in service communication	Tech Guide
Saga Pattern	Distributed transaction management	Tech Guide

How to Explain It Well

> **Opening**: "Microservices add complexity. I recommend starting monolithic and extracting services when you hit specific pain points."

> **Key talking points**:
>
> - "Monolith-first: faster to build, easier to refactor when domain unclear"
> - "Extract services when: team conflicts on deployment, different scaling needs"
> - "Strangler fig: incrementally extract, never big-bang rewrite"
> - "Microservices require: service discovery, distributed tracing, saga pattern"
> - "Conway's Law: architecture mirrors team structure"
